✨ Module 3

14 October 2026 20:55

Q18. Differentiate between One-Dimensional and Two-Dimensional Arrays in C with respect to their declaration, memory representation, and initialization.

Basis	One-Dimensional Array (1-D)	Two-Dimensional Array (2-D)
Declaration	`int arr[5];`	`int arr[2][3];`
Memory Representation	Elements are stored in a single linear sequence.	Elements are stored in rows and columns (matrix form).
Initialization	`int arr[5]= {10,20,30,40,50};`	`int arr[2][3]={{1,2,3}, {4,5,6}};`
Accessing Elements	Uses one index, e.g., `arr[2]`.	Uses two indices, e.g., `arr[1][2]`.

Conclusion:

- A **1-D array** stores data in a single row and uses one index.
- A **2-D array** stores data in rows and columns and uses two indices.

Q21. Explain any three built-in string handling functions available in the <string.h> header file, providing the exact syntax and purpose of each function.

String handling functions are predefined functions available in the `<string.h>` header file used to perform operations on strings.

1. strlen()

Syntax:

```c
strlen(string_name);
```

Purpose:

```
strlen(string_name);
```

Purpose:

Returns the length of a string (excluding the null character `\0`).

Example:

```c
strlen("Hello");   // Returns 5
```

2. strcpy()

Syntax:

```c
strcpy(destination, source);
```

Purpose:

Copies one string into another string.

Example:

```c
strcpy(str2, str1);
```

---

3. strcat()

Syntax:

```c
strcat(string1, string2);
```

Purpose:

Concatenates (joins) two strings by appending the second string to the first.

Example:

```c
strcat(str1, str2);
```

Q22. Write a C program that uses string manipulation functions from `string.h` to read two strings, concatenate them, and display the length of the resulting string.

Algorithm

1. Read two strings from the user.
2. Concatenate the strings using `strcat()`.
3. Find the length of the concatenated string using `strlen()`.
4. Display the concatenated string and its length.

Program

C

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char str1[50], str2[50];

    printf("Enter first string: ");
    scanf("%s", str1);

    printf("Enter second string: ");
    scanf("%s", str2);

    strcat(str1, str2);

    printf("Concatenated String = %s\n", str1);
    printf("Length = %d", strlen(str1));

    return 0;
}
```

Sample Output

```
Enter first string: Hello
Enter second string: World

Concatenated String = HelloWorld
Length = 10
```

What is an Array? Explain the Need for Arrays in C Programming.

Array:

An array is a collection of elements of the same data type stored in contiguous memory locations and accessed using a common name with an index.

Need for Arrays in C Programming

1. Arrays allow storing **multiple values of the same data type** under a single variable name.
2. They **reduce the number of variables** required in a program.
3. Arrays make **data processing and manipulation easier** using loops.
4. Elements can be accessed quickly using their **index position**.
5. Arrays help in **efficient memory management** as elements are stored contiguously.
6. They are useful for storing and processing large amounts of data such as **marks, salaries, or matrices**.

Example:

C

```c
int marks[5] = {80, 75, 90, 85, 88};
```

What is a String in C? Explain String Declaration and Initialization with Examples.

String:

A string in C is a collection of characters stored in a character array and terminated by a special null character (`'\0'`).

Answer in 6 Points

1. A string is used to store a sequence of characters such as names, words, or sentences.
2. In C, strings are stored in an array of type `char`.
3. **String Declaration:** A string is declared by specifying a character array.

```c
char name[20];
```

4. **String Initialization:** A string can be initialized at the time of declaration.

```c
char name[] = "Atharva";
```

5. Each string ends with a special null character (`'\0'`) which marks the end of the string.
6. Strings can be accessed and manipulated using functions available in the `<string.h>` header file.

Examples

Declaration:

```c
char city[20];
```

Initialization:

```c
char city[] = "Mumbai";
```